

第一次作业参考答案

▼ 第一次作业参考答案

▼ Problem 1

- (a)

- (b)

▪ Problem 2

▼ Problem 3

- (a)

- (b)

▪ Problem 4

▪ Problem 5

Problem 1

(1) 对于回归问题的线性回归模型，为什么损失函数选择为最小平方损失（预测值与真实值之间误差的均方误差）？优化目标是寻找合适的参数 $\mathbf{w}$ 最小化均方误差（MMSE），其背后的数学原理是什么？请给出数学推导过程？一般，可记最优模型参数为 $\mathbf{w}^*$ 或 $\mathbf{w}_o$ 。

该题共两小题，(a) 为分析题，(b) 为推导题。

(a)

线性回归的目标是尝试用一个线性函数来拟合给定数据点的分布。对于数据集 $\{x_i, y_i\}_{i=1}^N$ 中的一个点 (x_i, y_i) ，给出其估计值 $\hat{y} \equiv \mathbf{w} \cdot \phi(x)$ ，(其中 $\phi(x)$ 为特征函数向量，例如 N 次多项式特征向量 $\phi(x) = [1, x, x^2, \dots, x^n]^T$)，最直观的衡量二者误差的方式便是通过绝对值，即 L1 范数：

$$\mathcal{L}_{l1norm} \equiv \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| = |\mathbf{w}\phi(x_i) - y_i|$$

但该范数最致命的问题在于绝对值函数在零点处不可导，这限制了梯度下降等算法的发挥，导致模型无法训练。即使可以引入次梯度（subgradient），但显然更麻烦，且不在这门课的讨论范围内。

相较之下，将其平方得到的 L2 范数，不仅可以放大预测值与实际值之间的差异，其函数形式也处处可导且光滑，导数计算也十分简单。其次，L2 范数是一个严格的凸函数，这意味着它存在唯一的全局最优解，而不会被局部最优解干扰。导数计算的便利性也使得它往往可以直接计算出解析解。

若引入数据矩阵与向量 $\Phi \equiv [\phi(x_1), \phi(x_2), \dots, \phi(x_n)]^T$ ， $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$ ，则 L2 范数可以表示为

$$\mathcal{L}_{l2norm} \equiv \frac{1}{N} \|\Phi \mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{N} (\Phi \mathbf{w} - \mathbf{y})^T (\Phi \mathbf{w} - \mathbf{y})$$

利用矩阵求导的知识，令该范数导数为零，即可得到对应全局最优解的权重 $\mathbf{w}^*$

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}$$

(评分标准：扣分制，答到 L1 范数的不可导局限性，能够比较 L1 和 L2 范数的差别即给满分 0.2。)

后注：

1. 批改时发现许多同学似乎未能理解该题实际要考察的点为 L1 与 L2 范数的差别。题干表明的重点实则是对 MSE 自身函数性质（直观感性角度）的考虑。因此，对于该题的批改可以宽松给分，言之有理即可，但**必须作答**，第二问中的解答（数学原理角度）不能用于解答第一问。
2. 采用 MSE 的原因似有定理证明，Gauss-Markov Theorem，参见 [Wikipedia](#)

(b)

该过程的数学原理是**最大似然估计 (Maximum Likelihood Estimation, MLE)**。

假设真实值 y_i 与预测值 $\hat{y}_i = y_i + \epsilon_i$ 之间存在随机误差 ϵ_i ，通常认为误差 ϵ_i 是独立同分布的，且服从标准正态分布 $\epsilon_i = y_i - \hat{y}_i \sim \mathcal{N}(0, \sigma^2)$ 。

这时，考虑给定特征值 x_i ，则真实值 y_i 出现的概率为

$$p(y_i | x_i; \mathbf{w}) = \frac{1}{\sqrt{2\sigma}} \exp \left(-\frac{(y_i - \mathbf{w}x_i)^2}{2\sigma^2} \right)$$

为了求出最符合给定数据的 $\mathbf{w}$ ，需要进行最大似然估计，最大化所有样本出现的联合概率（此处取对数似然）：

$$\begin{aligned} l(\mathbf{w}) &= \log \left(\prod_{i=1}^N p(y_i | x_i; \mathbf{w}) \right) = \sum_{i=1}^N \log(p(y_i | x_i; \mathbf{w})) \\ &= \sum_{i=1}^N \left[\log \left(\frac{1}{\sqrt{2\sigma}} \right) - \frac{1}{2\sigma^2} (y_i - \mathbf{w}x_i)^2 \right] \end{aligned}$$

可以看到，若要最大化 $l(\mathbf{w})$ ，就要让第二项最小，若考虑括号外的求和符号，需要最小化的式子即为

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \sum_{i=1}^N (y_i - \mathbf{w}x_i)^2$$

此实际即为 L2 范数的等价形式。

(评分标准：扣分制，按点给分，答出数学原理为 MLE 得 0.1 分，给出真实值出现的概率 $p(y_i|x_i; \mathbf{w})$ 及其等价形式给 0.1 分，给出 $l(\mathbf{w})$ 的最终表达式得 0.1 分。)

Problem 2

(2) 对于线性回归模型，其损失函数采用均方误差 $\frac{1}{|D|} \sum_{(x,y) \in D} (f_{\mathbf{w}}(x) - y)^2$ ，即

$$\begin{aligned}\text{TrainLoss}(\mathbf{w}) &= \frac{1}{|D_{\text{train}}|} \sum_{(x,y) \in D_{\text{train}}} (f_{\mathbf{w}}(x) - y)^2 \\ &= \frac{1}{|D_{\text{train}}|} (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w}) \\ &= \frac{1}{|D_{\text{train}}|} \text{RSS}(\mathbf{w})\end{aligned}$$

基于矩阵运算表示，给出残差平方和 $\text{RSS}(\mathbf{w})$ 对模型参数 $\mathbf{w}$ 的梯度 $\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w})$ 的理论计算的推导过程，以及训练损失函数 $\text{TrainLoss}(\mathbf{w})$ 对模型参数 $\mathbf{w}$ 的梯度 $\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$ 的表示，并给出模型参数最优解的闭式解表达式。对下面 PPT 里的不妥之处，请指出。

本题共一小题，为推导题，任务是给出 $\mathbf{w}^*$ 解析解（见 Problem 1 (a)）的推导过程，该过程涉及矩阵求导相关知识。

最小化 TrainLoss 等价于最小化 RSS，因此只需要令 RSS 的梯度为零，即可得到对应的 $\mathbf{w}^*$ 。

根据矩阵求导的链式法则 $\nabla_{\mathbf{x}} f(g(\mathbf{x})) = \nabla_{\mathbf{x}} g(\mathbf{x}) \cdot \nabla_g f$ ，其中 $f(\cdot)$ 的值为常量，以及 $\nabla_{\mathbf{x}} (\mathbf{x}^T \mathbf{x}) = 2\mathbf{x}$ ， $\nabla_{\mathbf{x}} (\mathbf{A}\mathbf{x}) = \mathbf{A}^T$ ，可以得到

$$\begin{aligned}\nabla_{\mathbf{w}} (\mathbf{y} - \Phi \mathbf{w})^T (\mathbf{y} - \Phi \mathbf{w}) &= \nabla_{\mathbf{w}} (\mathbf{y} - \Phi \mathbf{w}) \cdot 2(\mathbf{y} - \Phi \mathbf{w}) \\ &= -\Phi^T \cdot 2(\mathbf{y} - \Phi \mathbf{w}) \\ &= -2\Phi^T (\mathbf{y} - \Phi \mathbf{w})\end{aligned}$$

令该式为零，当 $\Phi^T \Phi$ 可逆时，则有

$$\begin{aligned}-2\Phi^T (\mathbf{y} - \Phi \mathbf{w}^*) &= 0 \\ \Leftrightarrow \Phi^T \Phi \mathbf{w}^* &= \Phi^T \mathbf{y} \\ \Leftrightarrow \mathbf{w}^* &= (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y}\end{aligned}$$

PPT 中的不妥之处在于第一页底端处， $\nabla_{\mathbf{w}} \text{TrainLoss}(\mathbf{w})$ 的表达式少了一个负号。(PPT 抓虫可还行 awa)

(评分标准：扣分制，分点给分。正确给出 $\nabla_{\mathbf{w}} \text{RSS}(\mathbf{w})$ 的表达式给 0.1 分，求解 $\mathbf{w}^*$ 的逻辑正确给 0.1 分。PPT 的不妥之处就不计入评分了。)

Problem 3

(3) 实际应用中一般不选择闭式解，而是采用基于梯度下降的迭代优化算法寻找模型参数最优解 $\mathbf{w}^*$ ，为什么？写出最优模型参数 $\mathbf{w}^*$ 迭代计算的伪代码。

该题共两小题，(a) 为分析题，(b) 为伪代码。

(a)

可能的原因如下：

1. 计算复杂度：计算闭式解的时间复杂度约为 $\mathcal{O}(Nd^2 + d^3)$ ，其中 N 为数据点个数， d 为特征数量，当特征数量达到万级别或者更高（如文本分类、图像处理任务）时，计算闭式解的时间将变得极其漫长。相比之下，尽管批次梯度下降的计算速率远不如随机梯度下降，但它的时间复杂度约为 $\mathcal{O}(KNd)$ ，其中 $K \ll d^2$ 为算法收敛所需的迭代次数，因此计算复杂度远低于计算闭式解。
2. 进行矩阵计算时，需要将矩阵载入到内存中，无论是载入到 RAM 中，还是通过 GPU 并行计算载入到显存中，当特征数量以及数据量较大时，将矩阵完整载入通常会导致内存爆炸。虽然批次梯度下降同样不能解决这个问题，但随机梯度下降或小批量（Mini-batch）梯度下降只会加载部分数据进入内存，能有效避免该问题。
3. 闭式解的前提是 $\Phi^T \Phi$ 是可逆的，但该条件不一定能够得到保证。（例如样本数量 N 少于特征维数 d 时，根据矩阵乘法与矩阵秩的关系，即便 $\Phi \in \mathbb{R}^{N \times d}$ 为满秩矩阵（ $\text{rank } \Phi = N$ ）， $\Phi^T \Phi \in \mathbb{R}^{d \times d}$ 的秩也为 $N < d$ ，因此是不可逆的。）
4. 闭式解每次都使用当前所有的数据进行计算，若出现了新的数据，必须重新计算闭式解；梯度下降由于其迭代特性天然支持在线学习。
5. 闭式解只对形式简单的损失函数有效，但对于逻辑回归、神经网络等模型，根本不存在闭式解。统一使用梯度下降可以降低算法库的开发成本（一招鲜吃遍天？）

(评分标准：扣分制，五点之中答到两点即给满分 0.2，其它答案言之有理也可给分，但至少得说两点不然只给 0.1 分。)

(b)

参考伪代码如下，为简化可以将算法描述中的中文去除。

Algorithm 1 基于批量梯度下降的线性回归参数优化

输入: 特征矩阵 $\Phi \in \mathbb{R}^{N \times d}$, 标签向量 $\mathbf{y} \in \mathbb{R}^{N \times 1}$

学习率 $\alpha > 0$

最大迭代次数 $MaxIter$

收敛容差 ϵ

输出: 最优模型参数 $\mathbf{w}^* \in \mathbb{R}^{d \times 1}$

```
1: 随机初始化参数向量  $\mathbf{w}^{(0)}$  (如全零向量)
2: 初始化迭代计数器  $t \leftarrow 0$ 
3: while  $t < MaxIter$  do
4:   计算预测值:  $\hat{\mathbf{y}} = \Phi \mathbf{w}^{(t)}$ 
5:   计算当前梯度:  $\mathbf{g}^{(t)} = \frac{2}{N} \Phi^T (\hat{\mathbf{y}} - \mathbf{y})$ 
6:   if  $\|\mathbf{g}^{(t)}\|_2 < \epsilon$  then
7:     break {达到收敛条件, 提前跳出循环}
8:   end if
9:   更新参数:  $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \mathbf{g}^{(t)}$ 
10:  更新计数器:  $t \leftarrow t + 1$ 
11: end while
12: return 最终收敛的参数  $\mathbf{w}^* = \mathbf{w}^{(t)}$ 
```

(评分标准: 扣分制, 分点给分。明确输入和输出及它们的格式给 0.1 分; 包含计算预测值、计算梯度或等价部分给 0.1 分; 给出梯度下降的更新公式给 0.1 分。)

Problem 4

(4) 给出下面 PPT 页面里例子的模型参数迭代的详细计算过程。按规范的矩阵和向量运算规则给出。PPT 里给出了前两步, 请给出第 3 步和第 4 步的计算结果。

该题为计算题。计算思路如下:

1. 确认参与计算的变量: $\mathcal{D}_{train} : (x_1, y_1) = (1, 1), (x_2, y_2) = (2, 3), (x_3, y_3) = (4, 3);$
 $N = 3; \phi(x) = [1, x]^T; \alpha = 0.1$
2. 计算给定 $\mathbf{w}^{(i)}$ 时的预测误差 $e_j^{(i)} = (\mathbf{w}^{(i)} \cdot \phi(x_j) - y_j)$
3. 代入梯度公式

$$\nabla_{\mathbf{w}^{(i)}} \text{TrainLoss}(\mathbf{w}^{(i)}) = \frac{2}{N} \left(e_1^{(i)} \cdot \phi(x_1) + e_2^{(i)} \cdot \phi(x_2) + e_3^{(i)} \cdot \phi(x_3) \right)$$

4. 按下式更新参数得到 $\mathbf{w}^{(i+1)}$

$$\mathbf{w}^{(i+1)} \leftarrow \mathbf{w}^{(i)} - \alpha \cdot \nabla_{\mathbf{w}^{(i)}} \text{TrainLoss}(\mathbf{w}^{(i)})$$

计算得到答案如下：

$$\begin{aligned}\mathbf{w}^{(3)} &\approx [0.41, 0.93], & \nabla_{\mathbf{w}^{(3)}} \text{TrainLoss}(\mathbf{w}^{(3)}) &\approx [-1.64, -3.91] \\ \mathbf{w}^{(4)} &\approx [0.36, 0.70], & \nabla_{\mathbf{w}^{(4)}} \text{TrainLoss}(\mathbf{w}^{(4)}) &\approx [0.52, 2.33]\end{aligned}$$

（评分标准：扣分制，按点给分。梯度计算公式正确得 0.1 分，更新公式正确得 0.1 分；梯度结果和参数结果大致符合即得 0.1 分。）

Problem 5

(5) 以上内容，包括 PPT 页面，对于 $\text{TrainLoss}(\mathbf{w})$ 的表示似乎有两种不同的形式：

$$\begin{aligned}\text{TrainLoss}(\mathbf{w}) &= \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x, y) \in \mathcal{D}_{\text{train}}} (f(x) - y)^2 \\ &= \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x, y) \in \mathcal{D}_{\text{train}}} (\mathbf{w} \cdot \phi(x) - y)^2 \\ &= \frac{1}{|\mathcal{D}_{\text{train}}|} (\mathbf{y} - \Phi \mathbf{w})^\top (\mathbf{y} - \Phi \mathbf{w})\end{aligned}$$

区别是什么？有什么影响吗？

该题为简答题。这里有两种表示：标量求和表示与矩阵向量表示。二者在数学上是等价的，但在实际计算时存在很大差别：

1. 矩阵向量表示能够引入向量计算库（如 Numpy）和线性代数库来简化代码（标量表示通常需要编写 for 循环，但向量计算库已经内置了向量类和对应的运算函数），此外，矩阵向量表示还能利用现代 CPU 和 GPU 的并行计算能力（如 pytorch 的 GPU 运算支持），极大提高计算效率。
2. 数学求导上，矩阵向量表示的符号更简洁，求导也更方便（前提是记牢矩阵求导的公式和链式法则）

（评分标准：扣分制，答到矩阵向量和标量求和这两种表示给 0.1 分；答到矩阵向量表示能够并行化加速给 0.1 分。）